Scripting Shel

1. Che cos'è uno script?

Uno script è un file di testo con una sequenza di comandi che la shell esegue. È utile per automatizzare attività ripetitive, rendere una procedura riproducibile e ridurre gli errori manuali.

Bash è una shell comune su Linux. Lo script può essere avviato esplicitamente con `bash nome-file.sh`, oppure reso eseguibile e avviato come un programma.

Primo script

Creare `saluto.sh`:

```
#!/usr/bin/bash
printf 'Ciao dal mio primo script!\n'
```

La prima riga è lo **shebang**: indica al sistema quale interprete usare quando il file è eseguito direttamente. `/usr/bin/bash` individua Bash nell'ambiente dell'utente. Si può trovare in differenti cartelle in base alla distribuzione Linux o se su MacOS o su WSL2 in Windows 11.

Eseguire in uno dei due modi:

```
bash saluto.sh
```

oppure:

```
chmod u+x saluto.sh
./saluto.sh
```

`chmod u+x` aggiunge il permesso di esecuzione al proprietario del file. `./` dice alla shell di avviare il file che si trova nella directory corrente.

Commenti e output

Le righe che iniziano con `#` sono commenti, salvo la prima riga speciale dello shebang. Per stampare testo, `printf` è prevedibile e permette di controllare le nuove righe:

```
# Una nota per chi leggerà lo script
printf 'Operazione completata\n'
```

`echo` è più breve e va bene per esempi semplici; qui useremo soprattutto `printf` per ottenere un comportamento più uniforme.

1.2 Regole d'oro spazi

Sì, c'è una regola di fondo che spiega quasi tutto: **in bash gli spazi separano le parole (token), e la shell decide cosa fare in base alla posizione e alla forma di quelle parole**. Quindi lo spazio serve dove vuoi *separare* due elementi, e va evitato dove vuoi che restino *una cosa sola*.

1. Assegnazione di variabili: NIENTE spazi attorno a `=`

```
nome="Carlo"      # corretto
nome = "Carlo"    # errato: bash cerca di eseguire un coman
do chiamato "nome"
```

Il motivo: `nome=valore` deve essere un'unica parola perché la shell la riconosca come assegnazione.

2. Comandi e argomenti: spazi OBBLIGATORI

```
echo ciao      # corretto
echociao       # errato: comando inesistente
ls -l /tmp
```

3. Dentro `[ ]` e `[[ ]]`: spazi OBBLIGATORI

`[` è in realtà un **comando** (equivalente a `test`), e `]` è il suo ultimo argomento. Quindi vanno separati come qualsiasi altro argomento:

```
if [ "$a" = "$b" ]; then    # corretto
if ["$a"="$b"]; then        # errato
```

Vale anche per `[[ ... ]]`, e gli operatori (`=`, `-eq`, `-f`, ecc.) vogliono spazi da entrambi i lati.

4. Dentro `$(( ))`: gli spazi sono opzionali

Qui non si tratta più di parole di un comando ma di un'espressione aritmetica, quindi la shell è tollerante:

```
x=$((a+b))
x=$(( a + b ))   # equivalente, più leggibile
```

5. Dentro `$( )` e `{ }`: dipende

- `$(comando)`: spazi opzionali dopo `$(` e prima di `)`.
- `{ comandi; }` (gruppo di comandi): **spazio dopo `{` e `;` prima di `}` obbligatori**, perché `{` e `}` sono parole riservate.
- `${variabile}` (espansione): **niente spazi**, è una singola parola.

```
{ echo a; echo b; }     # corretto
{echo a; echo b;}       # errato
echo "${nome}"          # corretto
echo "${ nome }"        # errato
```

6. Operatori di controllo e redirezioni: di solito opzionali

`;`, `&&`, `||`, `|`, `>`, `<` sono riconosciuti anche senza spazi, perché sono caratteri speciali che la shell separa da sola:

```
ls|grep foo        # funziona
ls | grep foo      # più leggibile
echo hi>file.txt   # funziona
```

Attenzione però ai numeri di file descriptor: `2>&1` va scritto attaccato (`2> &1` non funziona).

7. Spazi dentro i valori: servono le virgolette

Se un valore *contiene* spazi e vuoi che resti un unico argomento, devi quotarlo:

```
cp "il mio file.txt" /tmp     # corretto
cp il mio file.txt /tmp       # errato: tre argomenti separ
ati
```

E per lo stesso motivo conviene quasi sempre scrivere `"$variabile"` tra virgolette, altrimenti il contenuto viene spezzato sugli spazi (word splitting).

Regola mnemonica

Chiediti: **"questa cosa è un comando con argomenti, o una singola parola/espressione speciale?"**

Situazione	Spazi
Comando e argomenti	obbligatori
Dentro `[ ]`, `[[ ]]`	obbligatori
`{ ...; }`	obbligatori
`var=valore`	vietati
`${var}`	vietati
`$(( ))`, `$( )`	opzionali
`|`, `;`, `&&`, `>`	opzionali

La maggior parte degli errori da principianti nasce da due casi: `var = valore` (spazi in più) e `[$a=$b]` (spazi in meno).

2. Variabili e virgolette

Assegnare una variabile non richiede spazi attorno a `=`. Per leggerla si antepone `$`:

```
nome="Ada"
printf 'Ciao, %s!\n' "$nome"
```

Le virgolette doppie preservano gli spazi contenuti nel valore. In generale, usare `"$variabile"` quando il contenuto rappresenta un singolo valore o un percorso:

```
cartella="Le mie note"
mkdir -p "$cartella"
```

Senza virgolette, spazi e caratteri speciali possono dividere il valore in più argomenti o attivare espansioni indesiderate. Le virgolette singole invece conservano il testo letteralmente:

```
printf '%s\n' '$nome'  # stampa $nome, senza sostituirlo
```

Per memorizzare l'output di un comando si usa `$(...)`:

```
oggi=$(date +%F)
printf 'Data: %s\n' "$oggi"
```

3. Argomenti e codici di uscita

Uno script può ricevere dati dalla riga di comando. `$1` è il primo argomento, `$2` il secondo; `$#` è il numero di argomenti e `"$@"` rappresenta tutti gli argomenti mantenendoli separati.

```
#!/usr/bin/env bash
printf 'Script: %s\n' "$0"
printf 'Primo argomento: %s\n' "${1:-nessuno}"
printf 'Numero di argomenti: %s\n' "$#"
```

`${1:-nessuno}` significa: usa il valore di `$1`, oppure `nessuno` se non è stato fornito o è vuoto.

Ogni comando restituisce un **codice di uscita**: per convenzione `0` indica successo, un valore diverso da zero un errore o un esito particolare. `$?` contiene il codice dell'ultimo comando:

```
grep -q 'Ada' rubrica.txt
esito=$?
printf 'Codice di uscita: %s\n' "$esito"
```

Spesso è meglio mettere direttamente il comando nella condizione di `if`, come vedremo sotto. `grep -q` cerca senza stampare le righe trovate; il risultato è

indicato dal suo codice di uscita.

4. Condizioni con `if`

```
if [[ -f "$1" ]]; then
    printf 'Il file esiste: %s\n' "$1"
else
    printf 'File non trovato: %s\n' "$1"
fi
```

`[[ ... ]]` è una condizione di Bash. `-f` verifica che il percorso esista e sia un file regolare; `-d` verifica una directory. Altri operatori utili:

- `e percorso`: esiste un file o una directory;
- `r percorso`: il percorso è leggibile;
- `z stringa`: la stringa è vuota;
- `stringa1 == stringa2`: le stringhe sono uguali;
- `numero1 -lt numero2`: confronto numerico “minore di”.

Per verificare se un comando riesce:

```
if grep -q 'errore' log.txt; then
    printf 'Trovata almeno una riga con “errore”.\n'
else
    printf 'Nessuna occorrenza.\n'
fi
```

`-q` serve a non mostrare le righe trovate

4.2 Bigino di `grep`

Forma generale:

```
grep [opzioni] 'testo-da-cercare' file
```

Per esempio:

```
grep 'errore' log.txt
```

Stampa le righe di `log.txt` che contengono `errore`. Per impostazione predefinita, la ricerca distingue maiuscole e minuscole e interpreta il testo cercato come un'espressione regolare di base.

Opzioni più usate

Opzione	Effetto	Esempio
`-i`	Ignora maiuscole e minuscole	`grep -i 'errore' log.txt`
`-v`	Mostra le righe che **non** corrispondono	`grep -v 'debug' log.txt`
`-n`	Mostra anche il numero di riga	`grep -n 'errore' log.txt`
`-c`	Conta le righe corrispondenti	`grep -c 'errore' log.txt`
`-l`	Mostra solo i nomi dei file con almeno una corrispondenza	`grep -l 'errore' *.log`
`-L`	Mostra i nomi dei file senza corrispondenze	`grep -L 'errore' *.log`
`-q`	Non stampa nulla: restituisce solo l'esito della ricerca	`grep -q 'errore' log.txt`
`-w`	Cerca una parola intera	`grep -w 'errore' log.txt`
`-x`	Cerca una riga intera uguale al testo	`grep -x 'OK' log.txt`
`-o`	Stampa solo la parte che corrisponde, non tutta la riga	`grep -o '[0-9]*' log.txt`
`-E`	Usa espressioni regolari estese	`grep -E 'errore
`-F`	Cerca testo letterale, senza interpretarlo come espressione regolare	`grep -F 'a.b' log.txt`
`-A N`	Mostra anche le `N` righe successive	`grep -A 2 'errore' log.txt`
`-B N`	Mostra anche le `N` righe precedenti	`grep -B 2 'errore' log.txt`
`-C N`	Mostra `N` righe prima e dopo	`grep -C 2 'errore' log.txt`
`-r`	Cerca ricorsivamente nelle cartelle	`grep -r 'errore' logs/`

- `A`, `B`, `C` sono disponibili nelle versioni comuni di `grep`; opzioni aggiuntive possono variare tra macOS e GNU/Linux.

Cercare più testi

Con `-E`, `|` significa “oppure”:

```
grep -E 'errore|warning' log.txt
```

Per cercare una stringa letterale che contiene caratteri speciali, usa `-F`:

```
grep -F 'file[1].txt' log.txt
```

Cercare in più file o da un comando

```
grep 'errore' *.log
```

```
cat log.txt | grep 'errore'
```

Di solito è più semplice scrivere:

```
grep 'errore' log.txt
```

Puoi anche passare un comando direttamente a `grep`:

```
comando | grep 'errore'
```

Usarlo in un `if`

`grep` restituisce un codice d’uscita:

- `0`: ha trovato una corrispondenza
- `1`: non ne ha trovate
- `2` o altro valore: si è verificato un errore

Per questo `-q` è comodo in un `if`:

```
if grep -q 'errore' log.txt; then
  echo "Trovato"
else
```

```
  echo "Non trovato"
fi
```

Le virgolette singole attorno al testo cercato sono una buona abitudine: evitano che la shell interpreti caratteri speciali.

5. Cicli

`for` : una lista di elementi

```
for nome in Ada Bruno Carla; do
    printf 'Ciao, %s!\n' "$nome"
done
```

Per elaborare file con un glob, tenere l'espansione tra virgolette quando si usa il singolo percorso, ma lasciare non quotato il pattern `*.txt` perché deve espandersi:

```
for file in ./*.txt; do
    [[ -e "$file" ]] || continue
    printf 'File: %s\n' "$file"
done
```

Il controllo `[[ -e "$file" ]] || continue` evita di trattare il pattern letterale `./*.txt` come se fosse un file quando non ci sono corrispondenze.

`while` : ripetere finché una condizione è vera

```
contatore=1
while (( contatore <= 3 )); do
    printf 'Passaggio %s\n' "$contatore"
    ((contatore += 1))
done
```

`(( ... ))` valuta un'espressione aritmetica in Bash. Si può anche leggere un file riga per riga:

```
while IFS= read -r riga || [[ -n "$riga" ]]; do
    printf 'Letta: %s\n' "$riga"
done < elenco.txt
```

`IFS=` impedisce di rimuovere spazi iniziali/finali; `read -r` non interpreta i backslash. La parte dopo `||` include l'ultima riga anche se il file non termina con una nuova riga.

6. Funzioni

Le funzioni raccolgono istruzioni riutilizzabili. I loro argomenti si leggono con `$1`, `$2`, come negli script:

```
saluta() {
    local nome=${1:-amica/o}
    printf 'Ciao, %s!\n' "$nome"
}

saluta "Ada Lovelace"
```

`local` limita la variabile alla funzione e riduce collisioni con altre variabili dello script. Una funzione può restituire un codice con `return`; per restituire testo, stampare il testo e catturarlo con `$(...)`.

7. Input dell'utente e scelta con `case`

```
printf 'Come ti chiami? '
IFS= read -r nome
printf 'Benvenuto/a, %s!\n' "$nome"
```

`case` è comodo quando ci sono più scelte testuali:

```
printf 'Scegli (s/n): '
IFS= read -r risposta
case "$risposta" in
    s|S) printf 'Hai scelto sì.\n' ;;
    n|N) printf 'Hai scelto no.\n' ;;
```

```
    *)   printf 'Risposta non riconosciuta.\n' ;;
esac
```

8. Errori e buone abitudini

- Mettere `#!/usr/bin/env bash` all'inizio degli script Bash.
- Mettere tra virgolette le variabili usate come percorsi o argomenti: `"$percorso"`.
- Usare nomi descrittivi per variabili e funzioni.
- Controllare gli argomenti richiesti e spiegare come usare lo script.
- Provare prima su una cartella temporanea e leggere i comandi prima di eseguirli.
- Evitare di eseguire script sconosciuti con privilegi amministrativi.
- `set -u` segnala variabili non definite; `set -o pipefail` fa fallire una pipeline se fallisce uno dei suoi comandi. Sono utili, ma richiedono di gestire con cura le variabili opzionali e i possibili errori.

Esempio di controllo esplicito degli argomenti:

```
#!/usr/bin/env bash

if [[ $# -ne 1 ]]; then
    printf 'Uso: %s NOME\n' "$0" >&2
    exit 2
fi

printf 'Ciao, %s!\n' "$1"
```

`>&2` invia il messaggio sullo standard error; `exit 2` termina lo script con un codice che segnala uso non valido.

Esercizi

Gli esercizi sono pensati per essere svolti in ordine. Per chi finisce in anticipo, gli esercizi 12–15 aggiungono casi più realistici.

1. Saluto personalizzato — facile

Modificare il primo script affinché chieda il nome con `read` e stampi `Ciao, NOME!`.

2. Due variabili — facile

Creare le variabili `citta` e `paese`, poi stampare una frase completa. Provare un valore che contenga uno spazio.

3. Data e directory — facile

Stampare la data nel formato `AAAA-MM-GG` e il percorso della directory corrente. Suggerimento: `date +%F` e `pwd`.

4. Argomento obbligatorio — facile

Scrivere `saluta.sh NOME`. Se manca l'argomento, mostrare una riga d'uso su stderr e terminare con un codice diverso da zero.

5. File presente? — facile

Scrivere uno script che riceve un percorso e comunica se esiste. Estenderlo per distinguere un file regolare da una directory.

6. Pari o dispari — facile/intermedio

Ricevere un numero intero e dire se è pari o dispari. Suggerimento: l'operatore `%` in `(( ... ))` calcola il resto.

7. Conto alla rovescia — facile

Stampare i numeri da 5 a 1 e poi `Via!`. Risolvere prima con `while`, poi con `for`.

8. Tabellina — intermedio

Ricevere un numero e stampare la sua tabellina da 1 a 10. Considerare che l'utente potrebbe non fornire un numero.

9. Elenco di file — intermedio

In una cartella di prova, contare quanti file `.txt` ci sono e stampare i loro nomi. Provare anche con una cartella senza file `.txt`.

10. Funzione di controllo — intermedio

Scrivere una funzione `controlla_file` che riceve un percorso e stampa `OK` se è leggibile, altrimenti `NON LEGGIBILE`.

11. Saluto multiplo — intermedio

Scrivere uno script che saluta ogni argomento ricevuto. Se non sono stati passati nomi, stampare un messaggio d'uso.

12. Ricerca in un file — intermedio

Ricevere un file e una parola; comunicare se la parola compare nel file. Gestire il caso in cui il file non esista. Suggerimento: `grep -q -- "$parola" "$file"`.

13. Menu testuale — intermedio

Mostrare un menu con tre opzioni (per esempio "data", "directory", "esci"), leggere la scelta e gestirla con `case`.

14. Leggere righe — intermedio

Leggere un file di nomi riga per riga e salutare ciascun nome. Provare anche con spazi e backslash nel contenuto.

15. Laboratorio: report della cartella — intermedio/avanzato

Creare uno script `report.sh` che riceve una directory e:

1. verifica che sia stata fornita e che sia una directory;
2. stampa il percorso della directory;
3. conta i file regolari `.txt` direttamente contenuti;
4. stampa i nomi trovati;
5. termina con un messaggio comprensibile anche se non trova file.

Provare il programma con una directory che contiene spazi nel nome e con una directory vuota.

Soluzioni di riferimento

Le soluzioni sono una possibile risposta, non l'unico modo corretto. Durante la correzione chiedere di spiegare perché le variabili dei percorsi sono quotate.

Soluzioni 1–5

```
# 1
#!/usr/bin/env bash
```

```
printf 'Come ti chiami? '
IFS= read -r nome
printf 'Ciao, %s!\n' "$nome"

# 2
citta="New York"
paese="Stati Uniti"
printf 'La città è %s, in %s.\n' "$citta" "$paese"

# 3
printf 'Data: %s\n' "$(date +%F)"
printf 'Directory: %s\n' "$(pwd)"

# 4
#!/usr/bin/env bash
if [[ $# -ne 1 ]]; then
    printf 'Uso: %s NOME\n' "$0" >&2
    exit 2
fi
printf 'Ciao, %s!\n' "$1"

# 5
#!/usr/bin/env bash
percorso=${1:-}
if [[ -z "$percorso" ]]; then
    printf 'Uso: %s PERCORSO\n' "$0" >&2
    exit 2
elif [[ -f "$percorso" ]]; then
    printf 'È un file regolare.\n'
elif [[ -d "$percorso" ]]; then
    printf 'È una directory.\n'
else
    printf 'Il percorso non esiste o non è un file/director
y ordinario.\n'
fi
```

Soluzioni 6–11

```
# 6: assume un intero fornito dall'utente
n=$1
if (( n % 2 == 0 )); then
    printf 'Pari\n'
else
    printf 'Dispari\n'
fi

# 7, while
n=5
while (( n >= 1 )); do
    printf '%s\n' "$n"
    ((n -= 1))
done
printf 'Via!\n'

# 7, for
for n in 5 4 3 2 1; do
    printf '%s\n' "$n"
done
printf 'Via!\n'

# 8: semplice validazione di intero con Bash
if [[ $# -ne 1 || ! $1 =~ ^-?[0-9]+$ ]]; then
    printf 'Uso: %s NUMERO_INTERO\n' "$0" >&2
    exit 2
fi
for ((i = 1; i <= 10; i++)); do
    printf '%s x %s = %s\n' "$1" "$i" "$(( $1 * i ))"
done

# 9
conteggio=0
for file in ./*.txt; do
    [[ -e "$file" ]] || continue
    ((conteggio += 1))
    printf '%s\n' "$file"
done
```

```
printf 'Totale: %s\n' "$conteggio"

# 10
controlla_file() {
    local percorso=$1
    if [[ -r "$percorso" ]]; then
        printf 'OK: %s\n' "$percorso"
    else
        printf 'NON LEGGIBILE: %s\n' "$percorso"
    fi
}
controlla_file "$1"

# 11
if [[ $# -eq 0 ]]; then
    printf 'Uso: %s NOME [NOME ...]\n' "$0" >&2
    exit 2
fi
for nome in "$@"; do
    printf 'Ciao, %s!\n' "$nome"
done
```

Soluzioni 12–15

```
# 12
#!/usr/bin/env bash
if [[ $# -ne 2 ]]; then
    printf 'Uso: %s FILE PAROLA\n' "$0" >&2
    exit 2
fi
file=$1
parola=$2
if [[ ! -f "$file" ]]; then
    printf 'File non trovato: %s\n' "$file" >&2
    exit 1
fi
if grep -q -- "$parola" "$file"; then
    printf 'Parola trovata.\n'
```

```
else
    printf 'Parola non trovata.\n'
fi

# 13
printf '1) Data\n2) Directory\n3) Esci\nScelta: '
IFS= read -r scelta
case "$scelta" in
    1) date +%F ;;
    2) pwd ;;
    3) printf 'Arrivederci!\n' ;;
    *) printf 'Scelta non valida.\n' ;;
esac

# 14
file=$1
if [[ ! -r "$file" ]]; then
    printf 'Uso: %s FILE_LEGGIBILE\n' "$0" >&2
    exit 1
fi
while IFS= read -r nome || [[ -n "$nome" ]]; do
    printf 'Ciao, %s!\n' "$nome"
done < "$file"

# 15
#!/usr/bin/env bash
if [[ $# -ne 1 || ! -d $1 ]]; then
    printf 'Uso: %s DIRECTORY_ESISTENTE\n' "$0" >&2
    exit 2
fi
directory=$1
conteggio=0
printf 'Report per: %s\n' "$directory"
for file in "$directory"/*.txt; do
    [[ -f "$file" ]] || continue
    ((conteggio += 1))
    printf '  %s\n' "${file##*/}"
```

```
done
printf 'File .txt trovati: %s\n' "$conteggio"
```